

Risposta alle domande G2 di Sistemi Operativi

Nicolò Giuliani

Indice

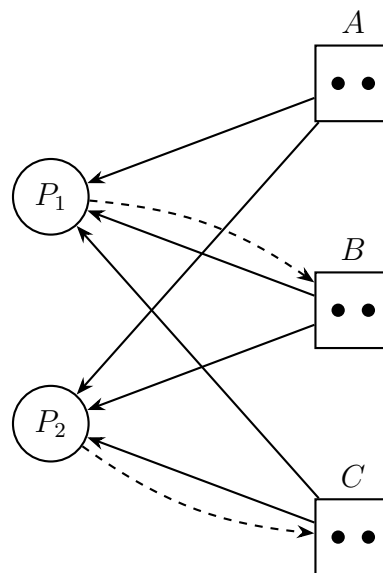
06 febbraio 2026	2
09 gennaio 2026	3
05 settembre 2025	4
21 luglio 2025	6
23 giugno 2025	7
28 maggio 2025	8
11 febbraio 2025	9
15 gennaio 2025	10
10 settembre 2024	11

- a) In una tabella FAT possono esistere elementi di uguale valore? (non considerando l'elemento nullo che contraddistingue la fine del file) Se sì: in quali casi? Se no: quali effetti avrebbe?

R: No, in FAT non possiamo avere elementi di uguale valore, infatti oltre a *EOF*, che serve per segnare il termine di una catena di puntatori. Se avessimo elementi di ugual valore vorrebbe dire che due puntatori indicano lo stesso blocco. Questo significherebbe che un blocco è stato sovrascritto da un altro file; ovvero uno o più file sono corrotti.

- b) Trovare, se possibile, un grafo di Holt che abbia 5 nodi, tutte le risorse abbiano molteplicità due (due unità per ogni classe di risorse) e tutti i processi siano in deadlock

R: Grafo di Holt con 2 processi + 3 classi di risorse (molteplicità 2) in deadlock.



- c) Perché i metodi di sincronizzazione tipo test&set (detti anche spinlock) assumono grande rilevanza nella scrittura di sistemi operativi multiprocessore?

R: Nei sistemi multiprocessore dobbiamo implementare il kernel in multiprocesso concorrente. Per non avere conflitti tra thread del kernel dobbiamo avere accesso atomico alle strutture dati del Kernel. Per far questo un semplice semaforo non basta dato che blocca a livello solo della singola CPU. Mentre con gli *spinlock*, ovvero istruzioni dell'IS della CPU, abbiamo accesso atomico su tutti i processori logici contemporaneamente.

- d) Perché è difficile revocare una autorizzazione fornita tramite capability?

R: Le capability non hanno una "sede centrale" dove possiamo gestirle tutte come per ACL, infatti ogni processo/utente (e anche i figli che la ereditano) ha la sua capability. Quindi in un sistema multiprocesso/multiutente diventa impossibile (o quasi) fare una revoca perché bisognerebbe scansionare l'intero disco (e sapere anche i figli).

a) Quali problemi possono accadere se si utilizza un file system ext2 dove un file ha un numero di riferimenti errato?

R: *Il numero di riferimenti in ext2 indica il numero di hard link del file, un numero errato comporta che abbiamo realmente piu' o meno hardlink.*

1. *Caso piu' hardlink: il fs segna che abbiamo piu' hardlink, quindi se andiamo a eliminare l'ultimo vero hardlink il fs non eliminerà internamente l'inode e non lascerà i blocchi come liberi, portando a un inutile memoria occupata per sempre.*

2. *Caso meno hardlink: il fs segna che abbiamo meno hardlink, quindi quando andiamo a eliminare un hardlink (anche se realmente ne abbiamo altri) il fs eliminerà l'inode e metterà i blocchi come liberi, portando a una corruzione del file.*

Per fixare entrambi i casi fsck (File System Consistency Check) scannerizza tutti i file e per ogni file conta il numero di hardlink reali e poi corregge se serve nel inode.

b) Siano dati due array di dischi identici, il primo viene gestito in modalità RAID1, il secondo in modalità RAID5. In quale array l'operazione di lettura è statisticamente più veloce?

R: *Il RAID1 ha come massimo di velocità in lettura la velocità di lettura minore di tutti i dischi. Mentre RAID5 avendo oltre che ridondanza anche uno striping abbiamo velocità in lettura maggiori rispetto anche al singolo disco.*

c) La seguente descrizione del funzionamento dell'algoritmo del banchiere è errata: "Quando un processo richiede risorse, l'algoritmo del banchiere controlla se lo stato del sistema rimarrebbe safe assegnando le risorse richieste: se sì le risorse vengono assegnate, se no l'operazione viene sospesa. Quando un processo restituisce risorse si controlla se il nuovo stato consente di soddisfare una o più operazioni sospese." Perché è errata? Come si può correggere la descrizione?

R: *Per la frase "... Quando un processo restituisce risorse si controlla se il nuovo stato consente di soddisfare una o più operazioni sospese." Bisognerebbe dire "... Quando un processo restituisce risorse si riesegue l'algoritmo del banchiere per vedere se il nuovo stato è safe o unsafe."*

d) A cosa serve e come funziona il meccanismo di Aging negli scheduler a priorità dinamica?

R: *Negli scheduler con priorità può accadere che i processi a bassa priorità non vengano mai eseguiti perché i processi ad alta priorità occupano sempre lo scheduler. Quindi per non far accadere questo con il passare del tempo i processi aumentano gradualmente la propria priorità fino a raggiungere in caso quella massima. Così tutti i processi verranno eseguiti.*

a) Quando è sconsigliato usare un meccanismo di protezione di accesso di tipo Access Control List?

R: *Se abbiamo bisogno di semplici permessi `r/w/x` per `owner/group/others` i permessi Unix bastano e le ACL aggiungono solo complessità e overhead. Per i servizi come docker o i file-sharing le ACL non funzionano. E rispetto le capability se vogliamo lavorare su processi e consentire ai processi singoli permessi speciali con capability possiamo mentre con ACL no.*

b) Qual è la differenza fra una funzione di una libreria ed una system call? In altre parole, perché sono necessarie le system call?

R: *La libreria generalmente sono fatte in:*

$$User\ space \rightarrow Syscall\ (kernel\ space) \rightarrow User\ space$$

*e quindi le librerie sono dei **wrapper** che ampliano le syscall.*

Abbiamo però sempre bisogno delle syscall infatti queste ci permettono di andare in kernel space per poter accedere alle strutture protette. Questo complessivamente è un meccanismo di protezione e di agevolamento al programmatore che seguendo la filosofia della OOP non deve preoccuparsi delle implementazioni interne.

c) Il problema del deadlock può essere risolto con tecniche di checkpointing & rollback?

R: *Sì, però questo consiste in snapshot periodici del sistema, rilevazione runtime del deadlock e ripristino del sistema (recovery). Questo porta come vantaggio il non dover conoscere a priori le risorse di ogni singolo processo rispetto al algoritmo del banchiere (preemption). Però porta un overhead di memoria per gli snapshot. Questa tecnica può essere ulteriormente ottimizzata nei linguaggi reversibili con modelli:*

```
try(condition){
    //do something that induce deadlock
} else{
    //rollback
}
```

Questo modello non porta overhead di memoria dal rollback dato che non abbiamo snapshot, ma porta a un costo di esecuzione maggiore perché rilevato un deadlock dobbiamo invertire tutte le operazioni per tornare alla situazione iniziale. Oltre al fatto che tutte le operazioni debbano essere invertibili per natura (a meno che di compilatori che traducono under-the-hood il linguaggio in uno reversibile per poi eseguirlo su interprete).

$$\begin{aligned} f(x) &\rightarrow f'(x) \\ f'(x)^{-1} &\rightarrow f(x) \end{aligned}$$

ovvero

$$f(x) \iff f'(x)$$

d) Dati N dischi aventi le stesse caratteristiche, il tempo medio di accesso è minore se creiamo con essi un cluster RAID0 o uno RAID5? Discutere la perdita di prestazione per la lettura e la scrittura.

R: Con RAID0 abbiamo N accessi, perché ogni file è diviso in striping su tutti i dischi. Mentre in RAID5 abbiamo 2 accessi ovvero disco dove scriviamo il blocco e il disco di parità'.

Lettura:

- RAID0: N accessi paralleli
- RAID5: 1 accesso

Scrittura:

- RAID0: N accessi paralleli
- RAID5: 4 accessi sequenziali

$$P_{new} = P_{old} \oplus D_{old} \oplus D_{new}$$

Con P e D rispettivamente per parità' e dato.

A livello di tempo medio il RAID0 vince.

21 luglio 2025

a) Perché durante la compattazione di memoria occorre sospendere l'esecuzione dei processi coinvolti?

R: *Perché i processi portano a frammentazione esterna e quindi si verifica un caso del genere:*

Prima: [P1][—][P2][—][P3]

Dopo: [P1][P2][P3][—]

Quindi la compattazione sposta i blocchi di memoria dei processi in una sequenza contigua per ottimizzare lo spazio totale. Però facendo questo gli indirizzi cambiano. Quindi il processo bisogna fermarlo, compattare, calcolare i nuovi indirizzi, applicarlo al processo.

b) Perché nell'i-node non è presente il nome del file?

R: *Nell'inode non memorizziamo il nome del file ma il contatore degli hardlink. Infatti se memorizzassimo il nome del file non avremmo la opzione degli hardlink (come per FAT), mentre attraverso il contatore degli hardlink questo è possibile. Il nome invece è memorizzato nella directory e anche il rispettivo inode.*

c) Perché il microkernel è meno efficiente del kernel monolitico? Perché è più flessibile e sicuro?

R: *Il mikrokernel avendo solo il minimo indispensabile in kernelspace e quasi tutti i driver, programmi, ecc in userspace per motivi di flessibilità, passa molto tempo a fare lo swap da usermode a kernelmode (perché richiamato molte volte) e quindi porta molto overhead. Però è decisamente più flessibile perché se vogliamo modificare un driver non dobbiamo ricompilare tutto il kernel ma solo il singolo programma. È anche più sicuro perché solo pochi componenti del SO possono accedere alle strutture protette del kernel, rendendo quindi la modifica/lettura molto più difficile.*

d) Nella memorizzazione da parte dei processi utente delle capability per il controllo di accesso può essere usato un algoritmo di crittografia a singola chiave (chiave privata) oppure occorre un algoritmo a doppia chiave (chiave pubblica)? Perché?

R: *Esclusivamente a chiave privata, perché il kernel firma la capability con la chiave privata (nascosta) e gli utenti/processi controllano la creazione sicura attraverso la loro pubblica. Questo serve perché se i processi/utenti conoscessero la chiave unica allora potrebbero creare loro una capability e firmarla con essa.*

23 giugno 2025

a) L'algoritmo LOOK (detto anche dell'ascensore) è più efficiente di C-LOOK (che ha lo stesso principio di funzionamento del metodo LOOK, ma la scansione del disco avviene in una sola direzione). In quali casi si preferisce C-LOOK a LOOK?

R: *LOOK ha un throughput maggiore ma il tempo tra le richieste agli estremi è molto alto, mentre C-LOOK va a risolvere il problema andando a minimizzare la varianza (differenza dei tempi medi).*

b) Spiegare quali attacchi sono possibili se non viene usato il meccanismo del "sale" (salt) nella memorizzazione delle password criptate.

R: *Il salt serve per proteggerci dagli attacchi dizionario. Infatti la password viene confrontata con $H(\text{password} + \text{salt})$ con salt come numero univoco per utente. questo numero è di 256bit\32byte.*

c) L'algoritmo del Banchiere evita il verificarsi di situazioni di deadlock ritardando le richieste che porterebbero il sistema in uno stato unsafe. Basta questa regola a evitare anche situazioni di starvation?

R: *No, infatti la richiesta potrebbe essere rimandata all'infinito. Bisognerebbe implementare anche una tecnica simile al aging per impedire starvation.*²

d) Perché è necessario usare spinlock in sistemi multiprocessore per implementare kernel di tipo simmetrico (SMP)?

R: Vedi risposta a pagina 2, esame 06 febbraio 2026.

a) Perché è difficile revocare un diritto di accesso se espresso come capability rispetto a revocare lo stesso diritto in una access control list?

R: *Perche' in capability non abbiamo una lista centrale per file/dir. come per ACL con tutti i permessi li, ma tante capability per diversi processi/utenti (e anche per i figli). Quindi la revoca diventa impossibile. Mentre per ACL ci basta modificare la tabella riferito a quel file.*

b) Si può implementare la memoria virtuale con paginazione a richiesta in un sistema che ha una Memory Management Unit priva di TLB (Translation Lookaside Buffer)?

R: *Si, pero' senza la TLB come buffer bisogna ogni volta leggere dalla tabella in RAM. Questo porta' un grande overhead.*

c) Perché non si può implementare la funzionalità di link fisico di file in un file system di tipo FAT?

R: *In FAT non esiste un reference counter associato ai blocchi: l'unica informazione presente è la catena nella tabella FAT. Se due directory entry puntassero forzatamente alla stessa catena, al momento della cancellazione di una delle due non ci sarebbe modo di sapere se esistono altri riferimenti, quindi i blocchi verrebbero liberati anche se l'altra entry li sta ancora usando. Per supportare gli hard link servirebbe un contatore di riferimenti, che FAT non prevede.*

d) L'algoritmo del Banchiere evita che si verifichino situazioni di deadlock. Sembra una funzione desiderabile eppure viene raramente usato nei sistemi reali, perché?

R: *Perche' nei sistemi reali e' quasi impossibile sapere a priori il numero di risorse che un algortimo ha bisogno, e quindi le condizioni per eseguire l'algortimo diventano impossibili. Poi c'e' anche da tenere in mente il costo computazionale alto perche' per n processi e r valute il costo e' di $O(n^2 \cdot r)$.*

11 febbraio 2025

a) La tecnica dell'aging serve per evitare casi di starvation. Come funziona?

R: *Si, la tecnica del aging serve per gli scheduler a priorita', nei quali se non avessimo aging alcuni processi potrebbero non essere mai eseguiti perche' i processi a priorita' maggiore occupano tutto il tempo la CPU. Quindi con il passare del tempo l'aging aumenta la priorita' di un processo che non e' stato eseguito.*

b) Può esistere un controller di device capace di usare DMA ma che non genera interrupt? Se NO: perché non esiste? Se SÌ: come deve essere costruito il relativo device driver?

R: *Puo' esistere ma allora in quel caso dovremmo avere la CPU che fa polling costantemente per vedere quando il trasferimento e' completato. Questo andrebbe ad annullare tutti i benefici del DMA. Quindi gli interrupt ci servono proprio per mettere in pausa la CPU e farla continuare con altri processi che non tocchino gli indirizzi che stiamo lavorando con il DMA.*

c) Quanti dischi sono coinvolti da una operazione di scrittura di un blocco in un array di dischi RAID 5? Perché?

R: *In scrittura in RAID5 abbiamo 4 operazioni sequenziali:*

- 1. Lettura blocco dati vecchio*
- 2. Lettura blocco parita' vecchio*
- 3. Scrittura dati nuovo*
- 4. Scrittura parita' nuova*

$$P_{new} = P_{old} \oplus D_{old} \oplus D_{new}$$

con P e D rispettivamente per parita' e dati.

d) Quale problema risolve la compattazione di memoria? Quali sono i difetti di questo metodo?

R: *La compattazione di memoria serve per ridurre la frammentazione esterna. Questa puo' essere fatta sia su dischi fissi, ma generalmente su RAM per i processi su CPU senza MMU. Se avessimo una MMU a priori la compatazione non servirebbe perche' ci basterebbe modificare la tabella degli indirizzi virtuali a indirizzi fisici. Il difetto principale e' che dobbiamo fermare i processi, spostare i dati, ricalcolare i nuovi indirizzi e copiarli nella PCB del processo e poi riprendere l'esecuzione.*

15 gennaio 2025

- a) Un i-node di un file system tipo ext2 per un errore viene riportato un numero di link maggiore del reale. Cosa può succedere se si continua ad usare il file system? (perché?) E se il numero di link errato fosse al contrario inferiore al reale cosa potrebbe succedere? (perché?)

R: Vedi risposta a pagina 3, esame 09 gennaio 2026.

- b) Perché è necessario usare spinlock in sistemi multiprocessore per implementare kernel di tipo simmetrico (SMP)?

R: Vedi risposta a pagina 2, esame 06 febbraio 2026.

- c) Perché nei sistemi reali l'algoritmo di rimpiazzamento second chance (orologio) viene preferito a LRU sebbene il primo non sia a stack e il secondo sì?

R: *Nei sistemi reali l'algoritmo LRU è troppo costoso da implementare perché per avere tempo di accesso alla domanda $O(1)$ dovremmo avere una tabella grande $n \times n$, mentre se ci accontentassimo di una risposta $O(n)$ ci basterebbe una vettore n che costantemente avrebbe aggiornato. Il costo di una memoria così veloce e di una grandezza così esponenziale è troppo alto in un sistema reale. Mentre per implementare **second chance** abbiamo $O(n)$ ma per la memorizzazione solo n bit. E questo algoritmo approssima LRU abbastanza precisamente per compensare.*

- d) Perché revocare un'autorizzazione espressa come capability è più difficile che revocare lo stesso diritto quando espresso come access control list?

R: Vedi risposta a pagina 8, esame 28 maggio 2025.

- a) In RAID61 (RAID1 di RAID6) qual è il livello di fault tolerance (quanti dischi possono guastarsi)? E in RAID16 (RAID6 di RAID1)?

R: In **RAID61** (RAID1 di RAID6) si mantengono due copie identiche di un array RAID6. Poiché RAID6 tollera fino a 2 dischi guasti per array, nel caso peggiore garantito si possono perdere **almeno 2 dischi qualsiasi**. Nel caso limite, se un intero array RAID6 cade, l'altro lo copre interamente, quindi si possono perdere tutti i dischi di un array più 2 dell'altro.

In **RAID16** (RAID6 di RAID1) i "dischi" del RAID6 sono coppie RAID1. RAID6 tollera la perdita di 2 array interi, e ogni coppia RAID1 tollera 1 disco guasto internamente. Quindi si possono perdere entrambi i dischi di al massimo 2 coppie (assorbiti dal RAID6), più un disco da ciascuna delle coppie rimanenti (assorbiti dal RAID1). La fault tolerance scala con il numero di coppie, rendendo RAID16 generalmente più robusto di RAID61.

- b) Nella configurazione di un sistema l'amministratore decide di creare una partizione di un disco rotazionale con file system FAT da montare come /tmp. È una buona o cattiva idea? Perché?

R: E' una cattiva idea perche' in FAT per leggere il blocco n dobbiamo prima leggere n blocchi e i file sotto /tmp chiedono molte letture veloci di blocchi in qualsiasi posizione. Oltretutto per gli HDD peggiorerebbero molto le performance.

- c) Quali sono i benefici della gestione degli interrupt nidificati e quali sono le difficoltà da affrontare per sviluppare un sistema operativo che supporti gli interrupt nidificati?

R: Il beneficio maggiore è la maggiore responsività del sistema, infatti per gli interrupt di I/O questi possono avere priorità maggiore rispetto ad altri in corso e questo porta a un sistema più responsivo per l'utente. Per far questo ci servirebbe uno stack per poter segnare a quale interrupt tornare, ripristinare il sistema a quando era stato bloccato e non avere cicli infiniti internamente.

- d) Perché aumentando la dimensione dell'area di memoria secondaria usata dalla memoria virtuale si corre il rischio di thrashing?

R: Perché la CPU accetterebbe l'esecuzione di più processi e quindi più memoria utilizzata e quindi dovremo passare sempre più tempo a passare dati da RAM a memoria secondaria, e avverrebbe thrashing peggiorando le performance.